

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра САПР

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Компьютерная Графика»
Тема: Формирование различных поверхностей с использованием ее
пространственного разворота и ортогонального проецирования на
плоскость при ее визуализации (выводе на экране дисплея)

Студенты гр. 3311	_____	Землякова С. А.
	_____	Кудрин П. В.
	_____	Сапронов К. Д.
Преподаватель	_____	Колев Г.Ю.

Санкт-Петербург
2025

Цель работы

Сформировать билинейную поверхность на основе произвольного задания ее четырех угловых точек. Обеспечить ее поворот относительно осей X и Y.

Теоретическая часть

В данной работе билинейная поверхность формируется на основе четырех угловых точек, которые определяют ее пространственную форму. Если соединить эти точки линиями, получится задающая рамка поверхности. Билинейная поверхность строится таким образом, что она проходит через все четыре угловые точки и плавно интерполирует пространство между ними.

Билинейная поверхность представляет собой частный случай параметрических поверхностей, где каждая точка поверхности вычисляется на основе двух параметров u и v , изменяющихся в диапазоне $[0,1]$. Поверхность строится на основе четырех угловых точек P_{00} , P_{10} , P_{01} , P_{11} , которые образуют пространственный четырехугольник.

Математическая модель билинейной поверхности описывается параметрическим уравнением:

$$P(u,v) = (1-u)(1-v) \cdot P_{00} + u(1-v) \cdot P_{10} + (1-u)v \cdot P_{01} + u \cdot v \cdot P_{11}$$

где:

- $u, v \in [0,1]$ - параметры поверхности
- $P_{00}, P_{10}, P_{01}, P_{11}$ - угловые точки в трехмерном пространстве
- Каждая точка $P(u,v)$ представляет собой взвешенную сумму угловых точек

Геометрическая интерпретация: поверхность образуется путем двойной линейной интерполяции - сначала вдоль одного параметра, затем вдоль другого. При фиксированном значении одного параметра получается прямая линия, что делает билинейную поверхность линейной по обоим направлениям.

Преобразования в пространстве осуществляются с помощью матриц поворота. Поворот вокруг осей X и Y вычисляется по формулам.

Поворот вокруг оси X:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) \\ 0 & \sin(\theta) & \cos(\theta) \end{bmatrix}$$

Поворот вокруг оси Y:

$$\begin{bmatrix} \cos(\theta) & 0 & \sin(\theta) \\ 0 & 1 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) \end{bmatrix}$$

Основные элементы управления:

1. Панель ввода координат - текстовые поля для задания координат четырех угловых точек (x, y, z)
2. Цветные метки точек - визуальное обозначение точек (P₀₀ - красный, P₁₀ - синий, P₀₁ - зеленый, P₁₁ - оранжевый)
3. Слайдеры поворота - регулировка угла поворота поверхности вокруг осей X и Y
4. Кнопка применения - обновление поверхности по введенным координатам
5. Кнопка сброса - возврат к начальным настройкам

Функциональные возможности:

1. Интерактивное задание точек - ввод координат через текстовые поля
2. Визуализация поверхности - построение билинейной поверхности с цветовой кодировкой высоты
3. Поворот поверхности - интерактивное вращение вокруг осей X и Y
4. Отображение угловых точек - цветные маркеры с подписями на графике

Пример работы программы.

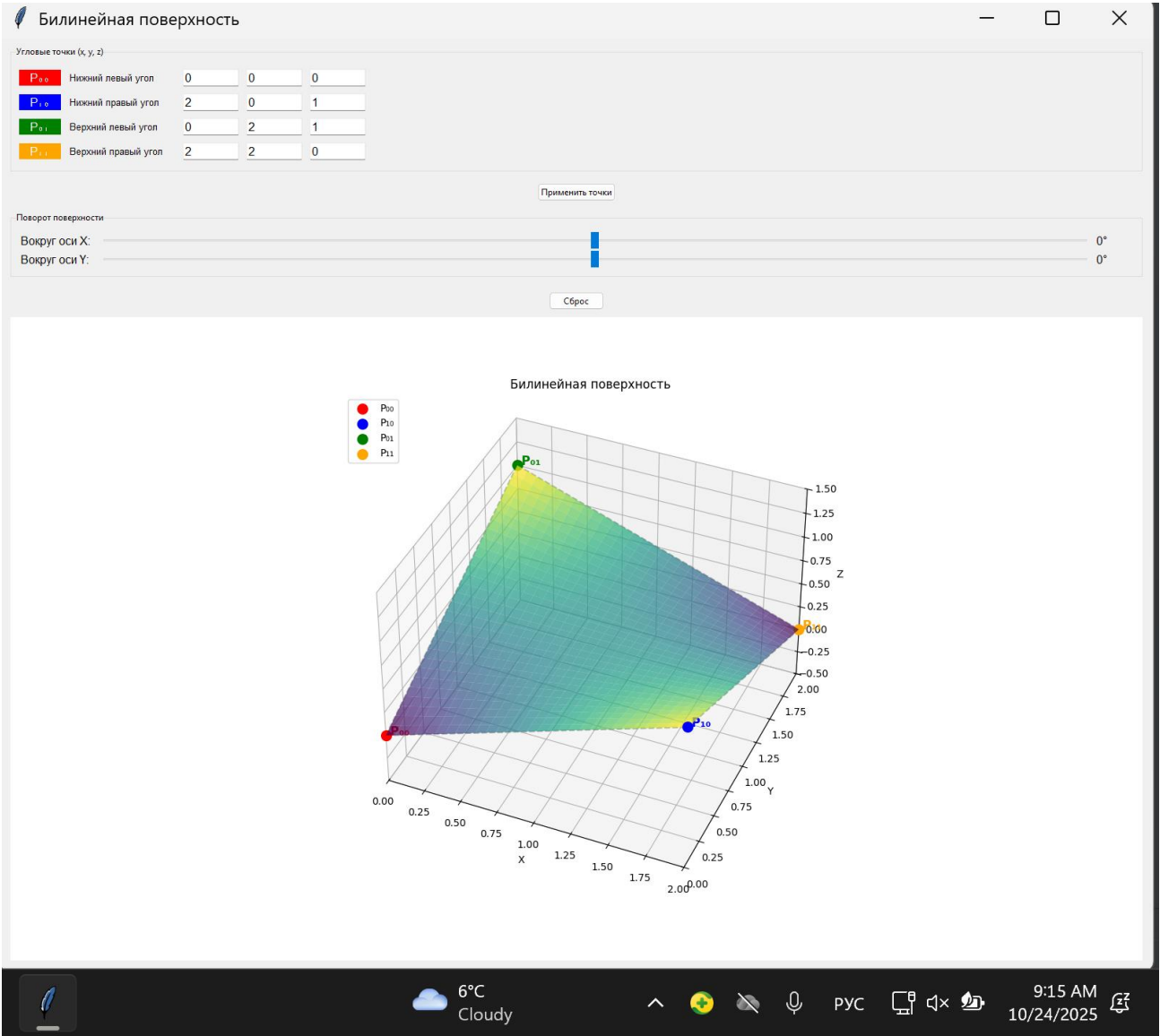


Рис. 1. Начальный интерфейс

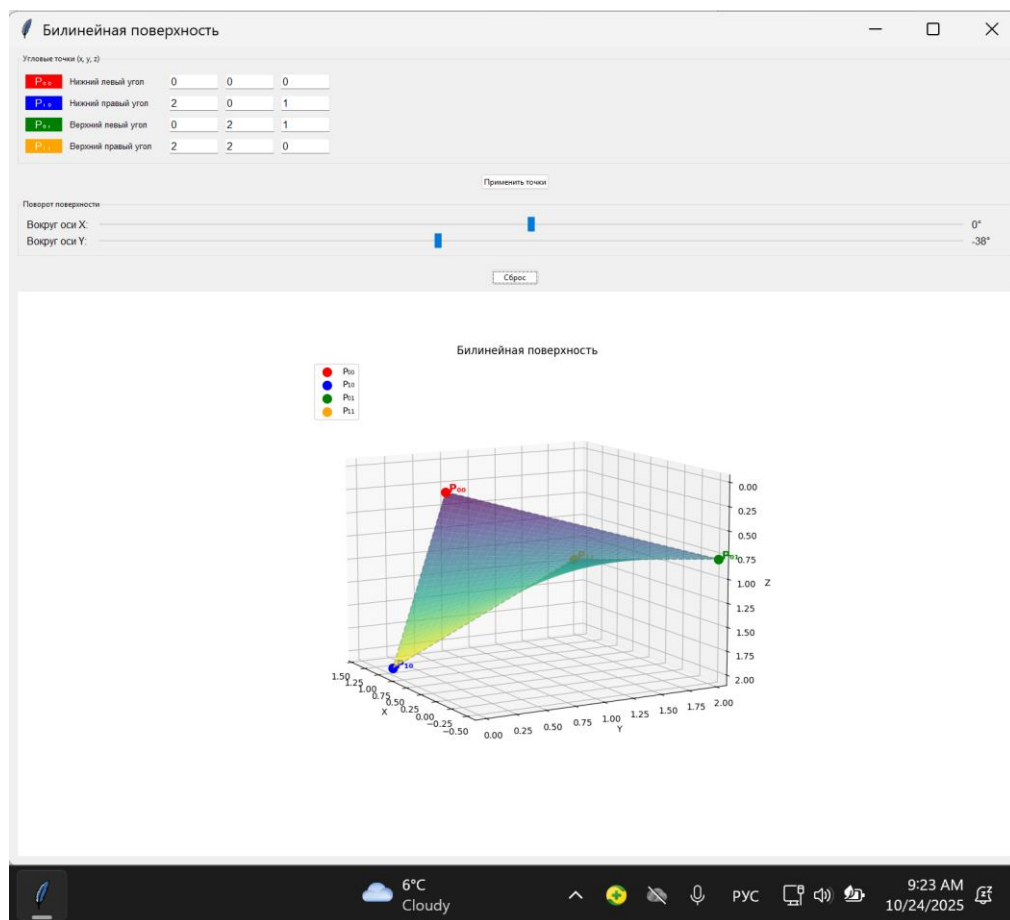
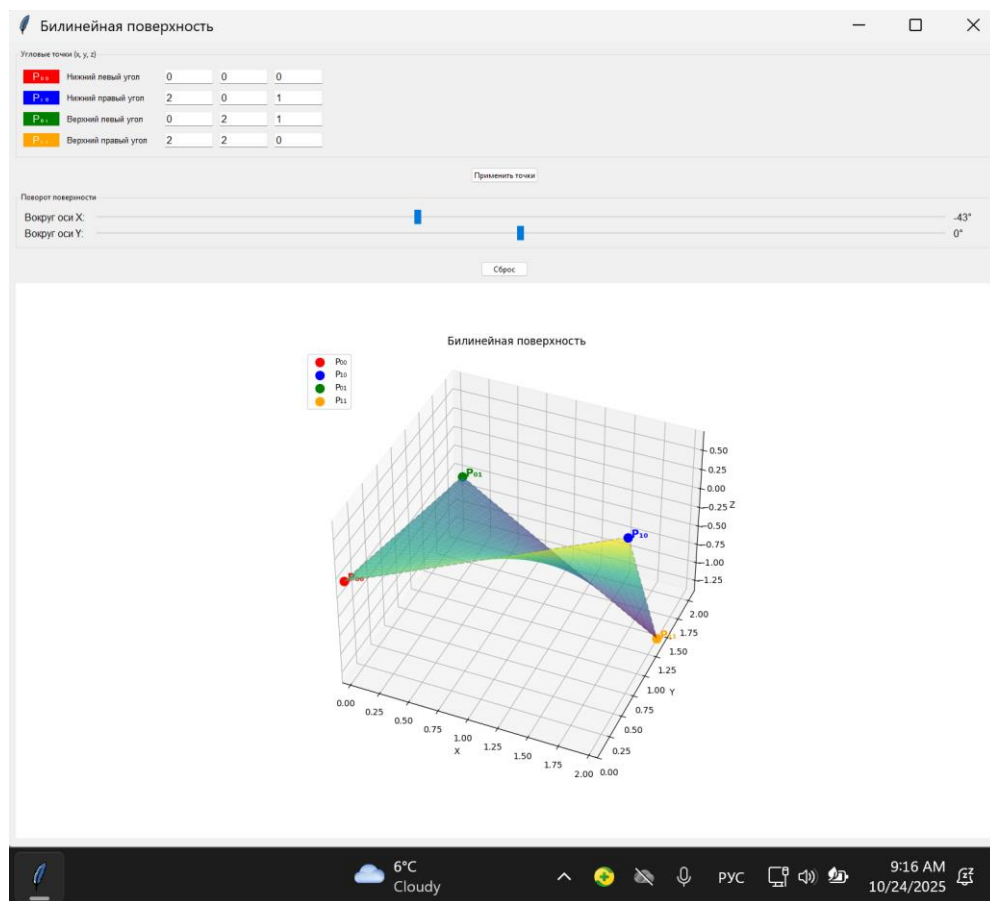


Рис. 2, 3. Повороты билинейной поверхности вокруг осей X и Y

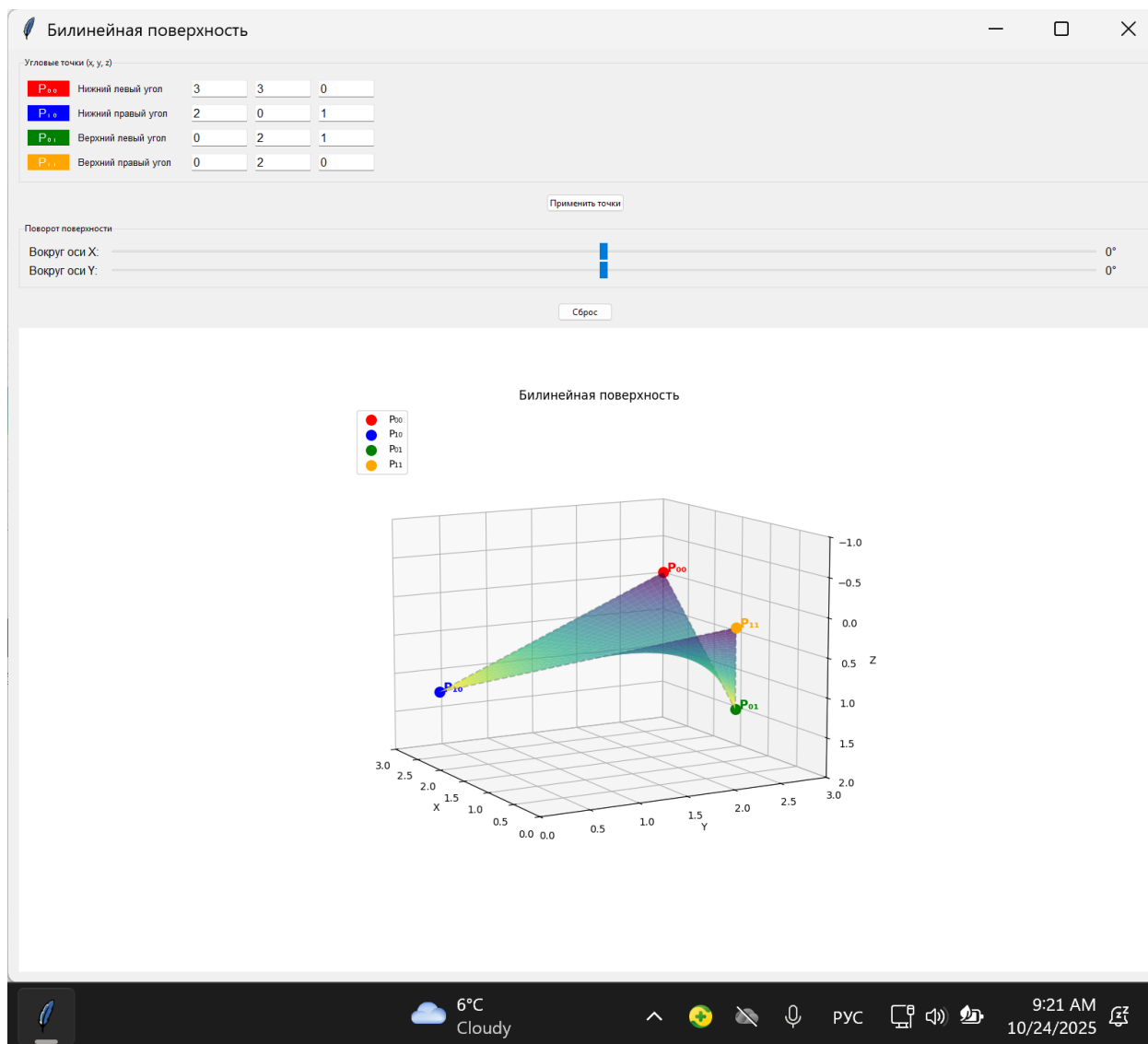


Рис. 4. Изменение координат точек

Выводы.

В ходе лабораторной работы была разработана программа для построения билинейных поверхностей в трехмерном пространстве. Основная цель работы - изучение математического аппарата билинейной интерполяции и его практическая реализация для формирования поверхностей.

Программа позволяет интерактивно работать с поверхностями: задавать координаты четырех угловых точек, наблюдать в реальном времени изменение формы поверхности при редактировании точек, а также вращать построенную поверхность вокруг осей X и Y для изучения ее пространственной структуры. Реализованный алгоритм корректно строит гладкие поверхности, которые проходят через все четыре угловые точки и плавно интерполируют пространство между ними.

Практическая работа подтвердила теоретические свойства билинейных поверхностей: линейность по обоим параметрическим направлениям, принадлежность выпуклой оболочке угловых точек и возможность точного задания граничных условий.

Исходный код.

```
import tkinter as tk
from tkinter import ttk
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

# Начальные точки
INIT_POINTS = [
    [0, 0, 0],
    [2, 0, 1],
    [0, 2, 1],
    [2, 2, 0]
]

class BilinearSurfaceApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Билинейная поверхность")
        self.root.geometry("1000x800")

        self.points = [list(p) for p in INIT_POINTS]
        self.angle_x = 0.0
        self.angle_y = 0.0

        # Цвета и метки для точек
        self.point_colors = ['red', 'blue', 'green', 'orange']
        self.point_labels = ['P00', 'P10', 'P01', 'P11']
        self.point_descriptions = [
            'Нижний левый угол',
            'Нижний правый угол',
            'Верхний левый угол',
            'Верхний правый угол'
        ]

        self.create_widgets()
        self.plot_surface()

    def create_widgets(self):
        # === Ввод точек ===
        points_frame = ttk.LabelFrame(self.root, text="Угловые точки (x, y, z)", padding=10)
        points_frame.pack(padx=15, pady=10, fill='x')

        self.entries = []
        large_font = ("Arial", 12)

        for i in range(4):
            row_frame = ttk.Frame(points_frame)
            row_frame.pack(fill='x', pady=5)

            # Метка точки с цветным фоном
            color_label = ttk.Label(row_frame, text=self.point_labels[i],
                                   font=large_font,
                                   width=6, background=self.point_colors[i],
                                   foreground='white', anchor='center')
            color_label.pack(side='left', padx=(0, 10))

            # Описание точки
            desc_label = ttk.Label(row_frame,
                                   text=self.point_descriptions[i],
                                   font=("Arial", 10), width=20, anchor='w')
```



```

desc_label.pack(side='left', padx=(0, 10))

point_entries = []
for j, coord in enumerate(['x', 'y', 'z']):
    entry = ttk.Entry(row_frame, width=8, font=large_font)
    entry.insert(0, str(self.points[i][j]))
    entry.pack(side='left', padx=5)
    point_entries.append(entry)
self.entries.append(point_entries)

# === Кнопка применения точек ===
apply_btn = ttk.Button(self.root, text="Применить точки",
command=self.apply_points)
apply_btn.pack(pady=5)

# === Слайдеры поворота ===
sliders_frame = ttk.LabelFrame(self.root, text="Поворот поверхности",
padding=10)
sliders_frame.pack(padx=15, pady=10, fill='x')

# Поворот вокруг X
ttk.Label(sliders_frame, text="Вокруг оси X:",
font=large_font).grid(row=0, column=0, sticky='w', padx=(0, 10))
self.slider_x = ttk.Scale(sliders_frame, from_=-180, to=180,
orient='horizontal',
command=self.on_rotate_x, length=300)
self.slider_x.set(0)
self.slider_x.grid(row=0, column=1, sticky='ew', padx=5)
self.angle_x_label = ttk.Label(sliders_frame, text="0°",
font=large_font, width=5)
self.angle_x_label.grid(row=0, column=2, padx=5)

# Поворот вокруг Y
ttk.Label(sliders_frame, text="Вокруг оси Y:",
font=large_font).grid(row=1, column=0, sticky='w', padx=(0, 10))
self.slider_y = ttk.Scale(sliders_frame, from_=-180, to=180,
orient='horizontal',
command=self.on_rotate_y, length=300)
self.slider_y.set(0)
self.slider_y.grid(row=1, column=1, sticky='ew', padx=5)
self.angle_y_label = ttk.Label(sliders_frame, text="0°",
font=large_font, width=5)
self.angle_y_label.grid(row=1, column=2, padx=5)

sliders_frame.columnconfigure(1, weight=1)

# === Кнопки сброса и обновления ===
btn_frame = ttk.Frame(self.root)
btn_frame.pack(pady=10)

reset_btn = ttk.Button(btn_frame, text="Сброс", command=self.reset)
reset_btn.pack(side='left', padx=10)

# === График ===
self.fig = plt.figure(figsize=(7, 5.5))
self.ax = self.fig.add_subplot(111, projection='3d')
self.canvas = FigureCanvasTkAgg(self.fig, master=self.root)
self.canvas.get_tk_widget().pack(fill='both', expand=True, padx=15,
pady=(0, 15))

for i in range(4):
    for entry in self.entries[i]:
        entry.bind("<Return>", lambda e: self.apply_points())

```

```

def get_points_from_entries(self):
    try:
        pts = []
        for i in range(4):
            x = float(self.entries[i][0].get())
            y = float(self.entries[i][1].get())
            z = float(self.entries[i][2].get())
            pts.append([x, y, z])
        return pts
    except ValueError:
        return self.points

def bilinear_surface(self, P00, P10, P01, P11, u_res=30, v_res=30):
    u = np.linspace(0, 1, u_res)
    v = np.linspace(0, 1, v_res)
    U, V = np.meshgrid(u, v)

    X = (1 - U) * (1 - V) * P00[0] + U * (1 - V) * P10[0] + (1 - U) * V *
P01[0] + U * V * P11[0]
    Y = (1 - U) * (1 - V) * P00[1] + U * (1 - V) * P10[1] + (1 - U) * V *
P01[1] + U * V * P11[1]
    Z = (1 - U) * (1 - V) * P00[2] + U * (1 - V) * P10[2] + (1 - U) * V *
P01[2] + U * V * P11[2]

    return X, Y, Z

def rotate_points(self, X, Y, Z, angle_x, angle_y):
    ax = np.radians(angle_x)
    ay = np.radians(angle_y)

    # Поворот вокруг X
    Y_rot = Y * np.cos(ax) - Z * np.sin(ax)
    Z_rot = Y * np.sin(ax) + Z * np.cos(ax)
    Y, Z = Y_rot, Z_rot

    # Поворот вокруг Y
    X_rot = X * np.cos(ay) + Z * np.sin(ay)
    Z_rot = -X * np.sin(ay) + Z * np.cos(ay)
    X, Z = X_rot, Z_rot

    return X, Y, Z

def plot_surface(self):
    self.ax.clear()

    P00, P10, P01, P11 = self.points

    X, Y, Z = self.bilinear_surface(P00, P10, P01, P11)
    Xr, Yr, Zr = self.rotate_points(X, Y, Z, self.angle_x, self.angle_y)

    # Рисуем поверхность
    self.ax.plot_surface(Xr, Yr, Zr, cmap='viridis', alpha=0.7,
edgecolor='none')

    # Рисуем точки с разными цветами и подписями
    rotated_points = []
    for point in self.points:
        # Поворачиваем каждую точку отдельно
        x, y, z = self.rotate_points(
            np.array([point[0]]),
            np.array([point[1]]),
            np.array([point[2]]),
            self.angle_x, self.angle_y
        )

```

```

        rotated_points.append([x[0], y[0], z[0]])

    # Отображаем точки с цветами и подписями
    for i, (point, color, label) in enumerate(zip(rotated_points,
self.point_colors, self.point_labels)):
        self.ax.scatter(point[0], point[1], point[2],
                        color=color, s=100, depthshade=True, label=label)
    # Добавляем подпись к точке
    self.ax.text(point[0], point[1], point[2], f' {label}',
                fontsize=12, color=color, fontweight='bold')

    # Рисуем рамку вокруг поверхности
    lines = [
        [rotated_points[0], rotated_points[1]], # P00 -> P10
        [rotated_points[0], rotated_points[2]], # P00 -> P01
        [rotated_points[1], rotated_points[3]], # P10 -> P11
        [rotated_points[2], rotated_points[3]] # P01 -> P11
    ]

    for line in lines:
        self.ax.plot([line[0][0], line[1][0]],
                    [line[0][1], line[1][1]],
                    [line[0][2], line[1][2]],
                    'gray', linestyle='--', alpha=0.6)

    self.ax.set_xlabel('X')
    self.ax.set_ylabel('Y')
    self.ax.set_zlabel('Z')
    self.ax.set_title('Билинейная поверхность', fontsize=12)

    # Легенда для точек
    self.ax.legend(loc='upper left', bbox_to_anchor=(0, 1))

    # Автоматическое масштабирование
    all_points = np.array(rotated_points)
    max_range = np.array([all_points[:, 0].max() - all_points[:,
0].min(),
                        all_points[:, 1].max() - all_points[:,
1].min(),
                        all_points[:, 2].max() - all_points[:,
2].min()]).max() / 2.0
    mid_x = (all_points[:, 0].max() + all_points[:, 0].min()) * 0.5
    mid_y = (all_points[:, 1].max() + all_points[:, 1].min()) * 0.5
    mid_z = (all_points[:, 2].max() + all_points[:, 2].min()) * 0.5
    self.ax.set_xlim(mid_x - max_range, mid_x + max_range)
    self.ax.set_ylim(mid_y - max_range, mid_y + max_range)
    self.ax.set_zlim(mid_z - max_range, mid_z + max_range)

    self.canvas.draw()

    def apply_points(self):
        new_points = self.get_points_from_entries()
        self.points = new_points
        self.plot_surface()

    def on_rotate_x(self, val):
        self.angle_x = float(val)
        self.angle_x_label.config(text=f"{int(self.angle_x)}°")
        self.plot_surface()

    def on_rotate_y(self, val):
        self.angle_y = float(val)
        self.angle_y_label.config(text=f"{int(self.angle_y)}°")
        self.plot_surface()

```

```
def reset(self):
    self.points = [list(p) for p in INIT_POINTS]
    for i in range(4):
        for j in range(3):
            self.entries[i][j].delete(0, tk.END)
            self.entries[i][j].insert(0, str(self.points[i][j]))
    self.slider_x.set(0)
    self.slider_y.set(0)
    self.angle_x = 0.0
    self.angle_y = 0.0
    self.angle_x_label.config(text="0°")
    self.angle_y_label.config(text="0°")
    self.plot_surface()

if __name__ == "__main__":
    root = tk.Tk()
    app = BilinearSurfaceApp(root)
    root.mainloop()
```